

Python

# Summary Session 6



# Преподаватель

Портрет

**Имя Фамилия**

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

# Важно

-  Камера должна быть включена на протяжении всего занятия
-  В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
-  Вести себя уважительно и этично по отношению к остальным участникам занятия
-  Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
-  Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя



# ПОВТОРЕНИЕ ИЗУЧЕННОГО



# Экспресс-опрос

?

Что такое методы строк?



## Методы строк

Для работы со строками доступен широкий набор встроенных методов. Эти методы позволяют выполнять различные операции.



## Методы проверки предиката

Это методы, которые возвращают True или False в зависимости от выполнения определённого условия для строки.

| Метод        | Описание                                                          |
|--------------|-------------------------------------------------------------------|
| isalpha()    | Проверяет, состоит ли строка только из букв.                      |
| isdigit()    | Проверяет, состоит ли строка только из цифр.                      |
| isalnum()    | Проверяет, состоит ли строка только из букв и цифр.               |
| isspace()    | Проверяет, состоит ли строка только из пробелов.                  |
| islower()    | Проверяет, все ли символы строки в нижнем регистре.               |
| isupper()    | Проверяет, все ли символы строки в верхнем регистре.              |
| istitle()    | Проверяет, начинается ли каждое слово в строке с заглавной буквы. |
| startswith() | Проверяет, начинается ли строка с указанной подстроки.            |



| Метод          | Описание                                                           |
|----------------|--------------------------------------------------------------------|
| endswith()     | Проверяет, заканчивается ли строка на указанную подстроку.         |
| isascii()      | Проверяет, состоит ли строка только из символов ASCII.             |
| isprintable()  | Проверяет, все ли символы строки являются печатаемыми.             |
| isidentifier() | Проверяет, является ли строка допустимым идентификатором в Python. |

# Пример



# проверяет, состоит ли строка только из букв (без пробелов и других символов)  
`print("Hello".isalpha())`

# проверяет, состоит ли строка только из цифр  
`print("12345.1".isdigit())`

# проверяет, состоит ли строка только из букв и цифр (без пробелов и специальных символов)  
`print("Hello123".isalnum())`

# проверяет, состоит ли строка только из пробельных символов (пробелы, табуляция и т.д.)  
`print(" ".isspace())`

# проверяет, находятся ли все символы строки в нижнем регистре  
`print("Hello".islower())`

# Пример



```
# проверяет, находятся ли все символы строки в верхнем регистре
print("HELLO".isupper())

# проверяет, начинается ли каждое слово в строке с заглавной буквы
print("Hello world".istitle())

# проверяет, начинается ли строка с указанной подстроки
print("Hello world".startswith("Hello"))

# проверяет, заканчивается ли строка указанной подстрокой
print("Hello world!".endswith("world"))
```

# Экспресс-опрос

?

Для чего нужен поиск подстроки в строке?

# Поиск подстроки в строке



Python предоставляет несколько способов для поиска подстроки в строке. Эти методы позволяют определить, содержится ли подстрока в строке, и найти её местоположение, если она присутствует, а также подсчитать количество вхождений подстроки.

| Метод    | Описание                                                                                          |
|----------|---------------------------------------------------------------------------------------------------|
| find()   | Возвращает индекс первого вхождения подстроки. Если подстрока не найдена, возвращает -1.          |
| rfind()  | Возвращает индекс последнего вхождения подстроки. Если подстрока не найдена, возвращает -1.       |
| index()  | Возвращает индекс первого вхождения подстроки. Если подстрока не найдена, вызывает ValueError.    |
| rindex() | Возвращает индекс последнего вхождения подстроки. Если подстрока не найдена, вызывает ValueError. |
| count()  | Возвращает количество вхождений подстроки в строке.                                               |

# Методы find() и rfind()



## find():

- Ищет первое вхождение подстроки и возвращает индекс её начала.
- Если подстрока не найдена, возвращает -1.

## rfind():

- Ищет последнее вхождение подстроки и возвращает индекс её начала.
- Если подстрока не найдена, также возвращает -1.

# Пример



```
text = "Python is awesome. Python is dynamic."
```

```
# Поиск первого вхождения подстроки
```

```
print(text.find("Python"))
```

```
# Поиск последнего вхождения подстроки
```

```
print(text.rfind("Python"))
```

```
# Подстрока не найдена
```

```
print(text.find("Java"))
```



# Методы `index()` и `rindex()`



## `index()`:

- Работает аналогично `find()`, возвращает индекс первого вхождения подстроки.
- Если подстрока не найдена, вызывает исключение `ValueError`.

## `rindex()`:

- Работает аналогично `rfind()`, возвращает индекс последнего вхождения подстроки.
- Если подстрока не найдена, также вызывает исключение `ValueError`.

# Пример



```
text = "Python is awesome. Python is dynamic."

# Поиск первого вхождения подстроки
print(text.index("Python"))

# Поиск последнего вхождения подстроки
print(text.rindex("Python"))

# Попытка найти отсутствующую подстроку вызывает ошибку
print(text.index("Java")) # ValueError
```

# Метод count()



Возвращает количество вхождений подстроки в строке, т.к. подсчитывает сколько раз подстрока найдена в строке. Этот метод всегда безопасен и не вызывает исключений.

# Пример



```
# Подсчёт количества вхождений подстроки
```

```
print(text.count("Python"))
```

```
# Если подстрока не найдена, возвращает 0
```

```
print(text.count("Java"))
```

```
# Следующую подстроку ищет после конца предыдущей, т.е. без пересечений
```

```
text = "hahahahahaha"
```

```
print(text.count("ha"))
```

```
print(text.count("haha"))
```

# Экспресс-опрос

?

Для чего нужны методы изменения регистра?

# Методы изменения регистра



В Python существуют несколько методов для работы с регистром строк. Эти методы позволяют преобразовывать символы строки в верхний или нижний регистр, а также обеспечивать правильное форматирование с заглавными буквами в начале каждого слова или строки.

| Метод        | Описание                                                                                        |
|--------------|-------------------------------------------------------------------------------------------------|
| upper()      | Преобразует все символы строки в верхний регистр.                                               |
| lower()      | Преобразует все символы строки в нижний регистр.                                                |
| capitalize() | Преобразует первую букву строки в заглавную, остальные символы делает строчными.                |
| title()      | Преобразует первую букву каждого слова строки в заглавную.                                      |
| swapcase()   | Меняет регистр всех символов строки на противоположный.                                         |
| casefold()   | Преобразует строку в нижний регистр с учётом языковых особенностей для более точного сравнения. |

# Пример



```
text = "Hello, world!"

# Преобразование строки в верхний регистр
print(text.upper())

# Преобразование строки в нижний регистр
print(text.lower())

# Первая буква строки становится заглавной, остальные строчными
print(text.capitalize())

# Первая буква каждого слова становится заглавной
print(text.title())

# Меняет регистр всех символов на противоположный
print(text.swapcase())

# Приводит строку к нижнему регистру с учётом особенностей языка
print("Straße".casefold())
```





## Метод `replace`

Используется для замены всех вхождений одной подстроки на другую в строке.

# Методы index() и rindex()



## Синтаксис

```
string.replace(old, new, count=-1)
```

## Разбор

- **old** — подстрока, которую нужно заменить.
- **new** — подстрока, на которую будет заменена старая подстрока.
- **count** — количество замен. Если не указано, заменяются все вхождения.

# Пример



# Пример замены всех вхождений

```
text = "I love Python, Python is great"
new_text = text.replace("Python", "Java")
print(new_text)
```

# Пример с ограничением замен

```
text = "apple apple apple"
new_text = text.replace("apple", "orange", 1)
print(new_text)
```



## Методы `split()` и `join()`

Используются для работы со строками, позволяя легко разделять строку на части и объединять список строк в одну строку с помощью разделителя.

| Метод                   | Описание                                                                                                     |
|-------------------------|--------------------------------------------------------------------------------------------------------------|
| split(sep, maxsplit=-1) | Разделяет строку по указанному разделителю и возвращает список строк. Можно ограничить количество разбиений. |
| join(iterable)          | Объединяет элементы последовательности в строку, используя указанный разделитель.                            |



## Метод `split()`

Разделяет строку на части по указанному разделителю и возвращает список этих частей.

# Методы `index()` и `rindex()`



## Синтаксис

```
string.split(sep, maxsplit=-1)
```

## Разбор

- **sep** — символ или подстрока, по которой происходит разделение.
- **maxsplit** — максимальное количество разбиений.

# Пример



# Пример 1: Разделение строки по пробелам

```
text = "apple banana cherry"
fruits = text.split()
print(fruits)
```

# Пример 2: Разделение строки по запятой

```
text = "apple,banana,cherry"
fruits = text.split(",")
print(fruits)
```

# Пример 3: Разделение строки с ограничением количества разбиений

```
text = "apple---banana---cherry"
fruits = text.split("---", 1)
print(fruits)
```





## Метод `split()`

Работает по-разному в зависимости от того, указан пробел как разделитель или нет.

# string.split():



## Синтаксис

```
text = "apple      banana\t cherry\n"

fruits = text.split()    # Разделяет по любым
                          пробельным символам

print(fruits)
```

## Разбор

- Если не указать разделитель, метод автоматически разделяет строку по любым пробельным символам.
- Кроме того, несколько подряд идущих пробелов будут восприниматься как один элемент.

# string.split(" "):



## Синтаксис

```
text = "apple      banana\t cherry\n"

fruits = text.split(" ") # Разделяет только по
                          пробелам

print(fruits)
```

## Разбор

- В этом случае метод разделяет строку только по пробелам.
- Если между словами несколько пробелов, они не будут восприниматься как один элемент, и это в результате создаст пустые строки.



## Метод join()

Объединяет элементы последовательности (списка, кортежа и т.д.) в одну строку, используя указанный разделитель.

# string.split(" "):



## Синтаксис

`separator.join(iterable)`

## Разбор

- **separator** — строка, которая будет использоваться в качестве разделителя между элементами.
- **iterable** — последовательность, элементы которой нужно объединить в строку.

# Пример



# Пример 1: Объединение списка строк с пробелом

```
fruits = ['apple', 'banana', 'cherry']  
text = " ".join(fruits)  
print(text)
```

# Пример 2: Объединение списка строк с запятой и пробелом

```
fruits = ['apple', 'banana', 'cherry']  
text = ", ".join(fruits)  
print(text)
```

# Пример 3: Объединение списка строк по пустому символу

```
litters = ['a', 'b', 'c', 'd', 'e']  
text = "".join(litters)  
print(text)
```

# Экспресс-опрос

?

Для чего нужны методы `strip`, `lstrip`, `rstrip`?



## Методы strip, lstrip, rstrip

Используются для удаления пробелов и других ненужных символов из строк.



| Метод                 | Описание                                                              |
|-----------------------|-----------------------------------------------------------------------|
| <code>strip()</code>  | Удаляет пробелы или указанные символы с обеих сторон строки.          |
| <code>lstrip()</code> | Удаляет пробелы или указанные символы только с левой стороны строки.  |
| <code>rstrip()</code> | Удаляет пробелы или указанные символы только с правой стороны строки. |



## Метод strip()

Удаляет пробелы или указанные символы с обоих концов строки (слева и справа).

# string.split(" "):



## Синтаксис

```
string.strip([chars])
```

## Разбор

- **chars (необязательный)** — символы, которые нужно удалить. Если не указано, по умолчанию удаляются пробелы.

# Пример



```
text = "\t hello world\n "  
  
print(text.strip()) # Удаляет все пробельные символы  
  
text_with_chars = "***hello***"  
  
print(text_with_chars.strip("*")) # Удаляет указанный символ  
  
text_with_chars = "**--*hello---**"  
  
print(text_with_chars.strip("*-")) # Удаляет все указанные символы
```



## Методы `lstrip()` и `rstrip()`

Удаляют пробелы или указанные символы только с левой или правой стороны строки соответственно.

# Пример



```
text = "  hello world  "

print(text.lstrip()) # Вывод: "hello world  "

text_with_chars = "***hello***"

print(text_with_chars.lstrip("*")) # Вывод: "hello***"

text = "  hello world  "

print(text.rstrip()) # Вывод: "  hello world"

text_with_chars = "***hello***"

print(text_with_chars.rstrip("*")) # Вывод: "***hello"
```

# Экспресс-опрос

?

Для чего нужно форматирование строк?



## Форматирование строк

Это способ вставки значений переменных или результатов выражений внутрь строки.



# Преобразование типов в C-style



В C-style форматировании можно легко преобразовывать числа в строки с помощью спецификаторов формата. Например, использование `%s` автоматически преобразует любое число в строку. Но при этом невозможно преобразовать строку в число, это вызовет ошибку `TypeError`.

# Пример



```
num = 42
```

```
text = "The number is %s." % num
```

```
print(text)
```

```
text = "42" # Число в формате str
```

```
formatted = "This will cause an error: %d" % text
```

```
print(formatted)
```

# Экспресс-опрос

?

Что такое методы изменения регистра?



## Метод `format`

Это гибкий способ форматирования строк, который позволяет вставлять значения в строку с использованием фигурных скобок `{}` в качестве плейсхолдеров.

# Метод format()



## Синтаксис

```
"строка с {} внутри".format(значение1,  
значение2, ...)
```

## Разбор

- Внутри строки используются фигурные скобки {}.
- Можно использовать позиционные аргументы, именованные аргументы.

# Позиционные аргументы



```
name = "Alice"  
age = 30  
text = "My name is {} and I am {} years old."  
print(text.format(name, age))
```

# Именованные аргументы



Каждому плейсхолдеру можно присвоить имя, что делает форматирование более понятным. При этом можно несколько раз использовать то же имя.

# Именованные аргументы



```
text = "My name is {name} and I am {age} years old. Are you also {age} years old?"  
  
print(text.format(name="Bob", age=25))
```



# Использование индексов



Можно явно указать, какой аргумент вставить в каждое место. Индексы также можно использовать несколько раз.

# Именованные аргументы



```
text = "Her name is {0} and she is {1} years old. {0} loves Python."  
print(text.format("Anna", 28))
```

# Комбинирование позиционных и именованных аргументов



Можно использовать как позиционные, так и именованные аргументы одновременно, что даёт большую гибкость в форматировании строк. При этом все позиционные аргументы указываются строго перед именованными.

# Комбинирование позиционных и именованных аргументов



```
text = "The {0} is {color}."  
print(text.format("sky", color="blue"))
```



## f-строки

Это удобный способ форматирования строк, который был введён в Python 3.6.

# f-строки



## Синтаксис

```
f"текст {переменная} текст {выражение}"
```

## Разбор

Чтобы использовать f-строки, перед строкой нужно добавить префикс `f`, а переменные или выражения помещаются напрямую в фигурные скобки `{}` внутри строки.

# Вставка переменных



```
name = "Alice"
age = 25
text = f"My name is {name} and I am {age} years old."
print(text)
```

# Вставка выражений



```
x = 10  
y = 20  
text = f"The sum of {x} and {y} is {x + y}."  
print(text)
```



# Вставка вызова функций и методов



```
text = "Python"

text_info = f"The length of '{text}' is {len(text)} and its uppercase version is {text.upper()}."

print(text_info)
```

# Вставка в многострочную строку



```
name = "Charlie"  
  
age = 30  
  
text = f"""Info  
  
Name: {name}  
  
Age: {age}  
  
"""  
  
print(text)
```

# Преимущества f-строк



**Удобство:** Позволяют вставлять переменные и выражения прямо в строку.



**Читаемость:** Делают код более понятным и лаконичным.



**Гибкость:** Поддерживают любые выражения внутри {}.



**Быстрота:** Работают быстрее, чем другие способы форматирования строк, такие как % и format().

# Экспресс-опрос

?

Что такое форматирование чисел?

# Форматирование чисел



Метод `format()` и f-строки предоставляют гибкие возможности для форматирования чисел. Эти два метода работают схожим образом, но f-строки более удобны и лаконичны.

# Основные способы форматирования чисел



Ограничение количества знаков после запятой: Используйте спецификатор `:.nf`, где `n` — это количество знаков после запятой, `f` — спецификатор для `float`.

# Пример



```
pi = 3.14159

# f-строки

text_fstring = f"Pi rounded to 2 decimal places is {pi:.2f}"

# Метод format()

text_format1 = "Pi rounded to 2 decimal places is {:.2f}".format(pi)

text_format2 = "Pi rounded to 2 decimal places is {0:.2f}".format(pi)

text_format3 = "Pi rounded to 2 decimal places is {num:.2f}".format(num=pi)

print(text_fstring)
print(text_format1)
print(text_format2)
print(text_format3)
```

# Форматирование с разделителями тысяч



Используйте спецификатор :, или :\_, чтобы добавить разделители тысяч в больших числах.



# Пример



```
large_number = 1234567890

# f-строки

text_fstring = f"The number with thousand separators: {large_number:,}"

# Метод format()

text_format = "The number with thousand separators: {:,} ".format(large_number)

print(text_fstring)

print(text_format)
```

# Выравнивание и ширина поля



И в методе `format()`, и в f-строках можно задавать выравнивание текста и чисел, а также ширину поля для их отображения. Это полезно при работе с форматированным выводом, например, в таблицах или структурированных данных.

# Основные спецификаторы



> — выравнивание по правому краю.



< — выравнивание по левому краю.



^ — выравнивание по центру.



Число после символом выравнивания задаёт минимальную ширину поля, которая будет выделена для значения.

# Выравнивание по правому краю



## Синтаксис

```
# f-строки
text_fstring = f"start_{'text':>10}_end"

# Метод format()
text_format = "start_{'>10'}_end"
print(text_fstring)
print(text_format.format("text"))
```

## Разбор

Чтобы выровнять значение по правому краю, используйте символ >.

# Выравнивание по левому краю



## Синтаксис

```
# f-строки
text_fstring = f"start_{'text':<10}_end"

# Метод format()
text_format = "start_{'<10}_end"
print(text_fstring)
print(text_format.format("text"))
```

## Разбор

Для выравнивания по левому краю используйте символ <.

# Выравнивание по центру



## Синтаксис

```
# f-строки
text_fstring = f"start_{'text':^10}_end"

# Метод format()
text_format = "start_{:^10}_end"
print(text_fstring)
print(text_format.format("text"))
```

## Разбор

Для выравнивания по центру используйте символ ^.

# Задание минимальной ширины поля



Можно задать только минимальную ширину для чисел, добавив число после символа двоеточия.



Для чисел выравнивание по умолчанию будет по правому краю.



Для строк выравнивание по умолчанию будет по левому краю.

# Пример



```
number = 40

text = 'hi'

# f-строки

text_fstring = f"start_{number:5}_end"

# Метод format()

text_format = "start_{:5}_end"

print(text_fstring)

print(text_format.format(text))
```



# Выравнивание чисел с заполнением



Python позволяет выравнивать строки и числа не только с помощью пробелов, но и с заполнением другими символами, например, нулями или любыми другими символами, которые вы укажете.



Для этого в f-строках можно указать символ заполнения перед символом выравнивания (<, >, ^) или до значения.

# Пример



# Заполнение нулями

```
number = 40
```

```
text = f"{number:0>5}"
```

```
print(text)
```

# Заполнение нижним подчеркиванием

```
text = f"{'Python':_ ^10}"
```

```
print(text)
```

# Методы выравнивания строк



В Python для выравнивания строк существуют три метода (`center`, `ljust`, `rjust`), которые помогают располагать строки по центру, слева или справа с использованием заданной ширины и, при необходимости, с заполнением оставшегося пространства символами.

# Методы выравнивания строк



`str.ljust(width[, fillchar])` — выравнивание строки по левому краю.



`str.rjust(width[, fillchar])` — выравнивание строки по правому краю.



`str.center(width[, fillchar])` — выравнивание строки по центру.

# Выравнивание по центру



## Синтаксис

```
str.rjust(width[, fillchar])
```

## Разбор

- **width** — минимальная ширина строки.
- **fillchar (опционально)** — символ для заполнения свободного пространства (по умолчанию — пробел).

# Примеры использования



```
text = "Python"
```

|                                       |          |              |    |        |      |
|---------------------------------------|----------|--------------|----|--------|------|
| #                                     | ljust(): | выравнивание | по | левому | краю |
| <pre>print(text.ljust(15))</pre>      |          |              |    |        |      |
| <pre>print(text.ljust(15, '-'))</pre> |          |              |    |        |      |

|                                       |          |              |    |         |      |
|---------------------------------------|----------|--------------|----|---------|------|
| #                                     | rjust(): | выравнивание | по | правому | краю |
| <pre>print(text.rjust(15))</pre>      |          |              |    |         |      |
| <pre>print(text.rjust(15, '-'))</pre> |          |              |    |         |      |

|                                        |           |              |    |        |
|----------------------------------------|-----------|--------------|----|--------|
| #                                      | center(): | выравнивание | по | центру |
| <pre>print(text.center(15))</pre>      |           |              |    |        |
| <pre>print(text.center(15, '-'))</pre> |           |              |    |        |



# ВОПРОСЫ





# РАЗБОР ДОМАШНЕГО ЗАДАНИЯ





# Разбор домашнего задания

## Звёздочки вместо чисел

Напишите программу, которая заменяет все цифры в строке на звёздочки \*.

```
text = "My number is 123-456-789"
```

## Пример вывода:

Строка: My number is 123-456-789

Результат: My number is \*\*\*-\*\*\*-\*\*\*

# Разбор домашнего задания

## Решение:

```
text = "My number is 123-456-789"

print("Строка:", text)

result = text

for char in text:
    if char.isdigit():
        result = result.replace(char, "*")

print("Результат:", result)
```

# Разбор домашнего задания

## Увеличение чисел

Напишите программу, которая делает все целые числа в строке в 10 раз больше.

```
text = "I have 5 apples and 10 oranges, price is 0.5 each. Card number is ....3672."
```

# Разбор домашнего задания

## Пример вывода:

I have 50 apples and 100 oranges, price is 5.0 each. Card number is ....3672.

# Разбор домашнего задания

## Пример решения:

```
text = "I have 5 apples and 10 oranges, price is 0.5 each. Card number is ....3672."
```

```
words = text.split()
```

```
for i in range(len(words)):
```

```
    if words[i].isdecimal():
```

```
        words[i] = str(int(words[i]) * 10)
```

```
new_text = " ".join(words)
```

```
print(new_text)
```

# Разбор домашнего задания

## Сумма положительных чисел

Напишите программу, которая обрабатывает список float чисел, вычисляет сумму положительных чисел.

Результат необходимо вывести в формате денежной суммы (до 2 символов после запятой), добавляя символ валюты "\$" вначале и разделяя тысячи запятой.

### Пример вывода:

```
numbers = [1245.4435, -302.1403, 87459.99, -520.8265, 1450.001]
```

### Пример вывода:

Сумма положительных чисел: \$90,155.43

# Разбор домашнего задания

## Пример решения:

```
numbers = [1245.4435, -302.1403, 87459.99, -520.8265, 1450.001]

# Инициализируем переменную для хранения суммы положительных чисел
positive_sum = 0

# Фильтруем положительные числа и суммируем их
for num in numbers:
    if num > 0:
        positive_sum += num

# Форматируем результат в денежном формате
formatted_sum = "${:,.2f}".format(positive_sum)

print(f"Сумма положительных чисел: {formatted_sum}")
```

# Разбор домашнего задания

## **Сумма положительных чисел**

Напишите программу, которая принимает список строк в формате "name age balance" и выводит информацию о каждом человеке с отформатированными данными: имя — 10 символов, возраст — 3 символа, баланс — 10 символов с двумя знаками после запятой. Используйте заранее подготовленный список строк.



# Разбор домашнего задания

## Данные:

```
data_list = [
    "John 23 12345.678",
    "Alice 30 200.50",
    "Bob 35 15000.3",
    "Charlie 40 500.75"
]
```

## Пример вывода:

|              |  |             |  |                  |
|--------------|--|-------------|--|------------------|
| Имя: John    |  | Возраст: 23 |  | Баланс: 12345.68 |
| Имя: Alice   |  | Возраст: 30 |  | Баланс: 200.50   |
| Имя: Bob     |  | Возраст: 35 |  | Баланс: 15000.30 |
| Имя: Charlie |  | Возраст: 40 |  | Баланс: 500.75   |

# Разбор домашнего задания

## Пример решения:

```
data_list = [
    "John 23 12345.678",
    "Alice 30 200.50",
    "Bob 35 15000.3",
    "Charlie 40 500.75"
]

for person in data_list:
    name, age, balance = person.split()
    age = int(age)
    balance = float(balance)

    print("Имя: {:<10} | Возраст: {:>3} | Баланс: {:>10.2f}".format(name, age, balance))
```

# Разбор домашнего задания

## Звёздочки вместо чисел

Напишите программу, которая принимает список строк в формате "name age balance" и выводит информацию о каждом человеке с отформатированными данными: имя — 10 символов, возраст — 3 символа, баланс — 10 символов с двумя знаками после запятой. Используйте заранее подготовленный список строк.

## Пример вывода:

```
data_list = [  
    "John 23 12345.678",  
    "Alice 30 200.50",  
    "Bob 35 15000.3",  
    "Charlie 40 500.75"  
]
```

# Разбор домашнего задания

## Пример вывода:

Имя: John | Возраст: 23 | Баланс: 12345.68

Имя: Alice | Возраст: 30 | Баланс: 200.50

Имя: Bob | Возраст: 35 | Баланс: 15000.30

Имя: Charlie | Возраст: 40 | Баланс: 500.75

# Разбор домашнего задания

## Решение:

```
data_list = [  
    "John 23 12345.678",  
    "Alice 30 200.50",  
    "Bob 35 15000.3",  
    "Charlie 40 500.75"  
]  
  
for person in data_list:  
    name, age, balance = person.split()  
    age = int(age)  
    balance = float(balance)  
  
    print("Имя: {:<10} | Возраст: {:>3} | Баланс: {:>10.2f}".format(name, age, balance))
```

## Заключение

